The Impact of Action Masking and Environmental Constraints on Rule Internalization in Transformer-Based Chess Agents

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

List of Figures

List of Tables

Glossary

$Q(s, a)$	The action-value function representing the mean expected reward for taking action a in state s.
$\pi(a\|s)$	The policy distribution; the probability of selecting action a given the current state s.
$V(s)$	The value function representing the scalar evaluation of a board position's win probability.
λ	The curriculum coefficient (illegal mass ratio) used to weigh policy vs. value loss.
$N(s, a)$	The visit count of an edge in the search tree, indicating how many times an action has been explored.
c_{puct}	A hyperparameter controlling the trade-off between exploration and exploitation in the search tree.
$A_{L(s)}$	The set of strictly legal moves available in state s according to FIDE rules.
$A_{P(s)}$	The set of pseudo-legal moves (geometrically valid but potentially leaving the king in check).
N_{select}	A node in the bipartite MCTS representing a board state where the agent must select a 'from' square.
N_{move}	A node in the bipartite MCTS where an origin square has been fixed and the agent selects a 'to' square.
K_{cap}	The terminal event of king capture used as the reward signal in the pseudo-legal rule set.
d_{model}	The hidden dimensionality of the Transformer encoder's internal representations.

Abbreviations

MCTS	Monte Carlo Tree Search
PUCT	Predictor + Upper Confidence Bound applied to Trees
RL	Reinforcement Learning
MDP	Markov Decision Process
FIDE	Fédération Internationale des Échecs (International Chess Federation)
CNN	Convolutional Neural Network
TD	Temporal Difference (Learning)
FIFO	First-In, First-Out (Replay Buffer)
SOTA	State Of The Art
MSB / LSB	Most Significant Bit / Least Significant Bit
W/D/L	Win / Draw / Loss
FEN	Forsyth-Edwards Notation

Chapter 1

Introduction

1.1 Project Background

Chess is a deterministic, perfect-information zero-sum game played on an 8×8 grid. Each player commands 16 pieces of 6 distinct classes, each governed by strict kinematic rules. The objective is to trap the opponent's king (checkmate). Historically dubbed the "Drosophila of AI" McCarthy (1990), chess provides an optimal substrate for studying machine intelligence: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space ($\approx 10^{40}$ states).

Modern machine learning frequently decouples models from environment transition dynamics via **Logit Masking**, a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While computationally efficient, this deprives the network of negative feedback for invalid predictions, bypassing the model's need to internally represent the mechanics of play.

As AI systems scale toward generalized world models, the ability of an agent to internalize the causal rules of its environment (as opposed to hardcoded environmental constraints) becomes critical. Understanding how neural networks represent physical constraints versus strategic heuristics is fundamental to interpretability.

Literature has largely omitted the impact of removing these guardrails because standard reinforcement learning (RL) prioritizes asymptotic Elo gain per Floating Point Operation. Masking is a known optimization that accelerates convergence (Huang and Ontañón (2020)). Consequently, State Of The Art (SOTA) engines (e.g., AlphaZero - David Silver, Hubert, *et al.* (2017)) utilize hardcoded legal move generators, leaving the representational dynamics of rule acquisition from scratch in complex Markov Decision Processes (MDP) unexplored. This project aims to investigate the learning dynamics and internal representations that emerge when these guardrails are removed.

1.2 Project Description

Two orthogonal axes of environmental constraint are examined. Crucially, these axes isolate distinct learning targets: **Action Masking** dictates the policy head's acquisition

of piece kinematics, while **Environmental Constraints** dictate the value head's acquisition of survival heuristics.

A custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Search Tree (MCTS) are introduced to facilitate explicit measurement of model interpretability. This architecture decouples piece selection from destination selection to provide a granular window into the model's spatial reasoning.

The experimental configurations are defined across two axes:

1. **Axis 1: Rule Set Convergence (Action Space Constraints)**
   - **Control:** Training on a strict legal ruleset.
   - **Test:** Training on a pseudo-legal ruleset where the terminal state is the simpler, denser king capture $\left(K_{\text{cap}}\right)$ over the highly sparse, complex $(A_l = \emptyset)$. This variant of Chess fundamentally alters the Markov Decision Process (MDP) reward function $R(s, a)$ yielding a potentially smoother reward landscape, forcing the value head to learn survival heuristics (e.g., pins) rather than relying on hardcoded environment evaluations.
   - **Hypothesis:** The network first bootstraps learning the rules and goals of the environment. Ideal play in the pseudo-legal space A_P may converge to standard Chess where concepts such as pins and checkmates emerge as survival heuristics to prevent or secure king capture. Conversely, the removal of environmental guardrails may degrade MCTS value estimation, preventing policy bootstrapping.
2. **Axis 2: Logit Masking**
   - **Control:** Masked logits (illegal moves are mathematically eliminated prior to selection).
   - **Test:** Unmasked logits (illegal moves are permitted by the network but penalized by the environment/loss function).
   - **Ablation:** Unmasked logits with a fixed policy/value loss ratio, isolating the impact of the proposed self-annealing curriculum loss (Section 3.6).
   - **Hypothesis:** Unmasked training will incur an initial sample-efficiency penalty by introducing the dual-optimisation problem of learning game mechanics alongside strategy, but may lead to more robust internal representations reflecting potentially stronger play long term.

1.3 Scope and Limitations

Several limitations bound the scope of the work:

1. The comparison of interest is across five training configurations over asymptotic performance. As such, models are not benchmarked against external engines; absolute Elo against SOTA requires significantly more compute than the project budget allows.
2. The value head is bootstrapped against MCTS root values over terminal outcomes to accelerate convergence, constrained by GPU throughput.

1.4 Aims and Objectives

The primary aim is to train multiple transformer-based agents via self-play reinforcement learning across the varying rule sets and action spaces and to quantitatively evaluate their performance, learning trajectories, and emergent behaviours.

Specific objective include:

- **Implement high-throughput chess library:** Develop a dual ruleset engine optimized for RL self-play.
- **Architect action-selection models:** Design a two-pass autoregressive transformer and bipartite MCTS to model piece $\rightarrow$ square selection.
- **Execute comparative training:** Train five configurations, varying move legality (Legal/Pseudo-Legal), masking, and loss-annealing, using identical hyperparameters.
- **Quantify performance metrics:** Measure convergence rates, illegal-move frequencies, and the evolution of rule-abiding behavior in unconstrained models.
- **Analyze spatial reasoning:** Correlate the emergence of legal play with attention distribution patterns in the two-pass encoder.

1.5 Thesis Structure

The remainder of this dissertation is structured as follows:

- **Chapter 2** surveys the literature surrounding reinforcement learning, Transformer architectures, action masking, and curriculum learning, identifying the gap this work addresses.
- **Chapter 3** details the system design, including the bipartite MCTS, the two-pass encoder, and the self-annealing loss function.
- **Chapter 4** outlines the implementation of the Rust engine and training pipeline, with particular attention to the engineering decisions that make high-throughput self-play tractable.
- **Chapter 5** presents the results and evaluation across the four configurations.
- **Chapter 6** concludes and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as an MDP

Chess can be formally modelled as a MDP defined by the tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{40}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon (1950), Andrew and Richard S (2018)). Standard implementations enforce a legal action space $A_{L(s)} \subset A$ in which moves leaving the king in check are excluded by the environment World Chess Federation (FIDE) (2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$ such that $A_{L(s)} \subset A_{P(s)} \subset A$, permitting all geometrically valid piece movements and shifting the terminal condition from the abstract *checkmate* to the explicit *king capture* K_{cap}.

2.2 Chess Played by Machines

2.2.1 AlphaZero and MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (David Silver, Hubert, *et al.* (2017)), which combined MCTS with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

MuZero (Schrittwieser *et al.* (2020)) removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts. The literature lacks insight into how the removal of environmental guardrails or action masking impacts the sample efficiency and representational robustness of the underlying network.

In domains with highly structured action spaces, such as StarCraft II, AlphaStar (Vinyals *et al.* (2019)) demonstrated the efficacy of autoregressive policy heads to factor complex joint distributions. Adapting this to chess, factoring the joint probability as

$$P(a|s) = P(\text{from}|s) \cdot P(\text{to}|s, \text{from})$$

collapses the output dimensionality from $d_{\text{model}} \times 4672$ to $d_{\text{model}} \times 64$ and gives rise to two distinct, observable failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

While both AlphaZero and MuZero proved the viability of MCTS + RL, their success is reliant on the embedded spatial CNNs biases that Transformers remove, making Transformers a superior substrate for studying pure rule internalization.

2.2.2 Transformers in Chess

Recent advancements demonstrate the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. Monroe and Chalmers (2024) achieved Grandmaster-level performance with a Transformer relying purely on attention to evaluate board states, without explicit search. Jenner *et al.* (2024) provided complementary evidence of learned look-ahead in chess-playing networks, showing that attention heads encode candidate future board states. McGrath *et al.* (2022) further demonstrated that AlphaZero-style networks acquire human-interpretable concepts (material balance, mobility, king safety) over the course of training.

These results frame the central architectural choice of the present work: decoupling $P(\text{from}|s)$ from $P(\text{to}|s, \text{from})$ provides an observable window into distinct failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i|s) = \frac{\exp(z_i + M_i)}{\sum_j \exp\big(z_j + M_j\big)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

Huang and Ontañón (2020) demonstrated that collapsing the exploration space via action masking significantly improves sample efficiency. However, logit masking prevents the network receiving feedback from masked indices due to gradients being nullified in the compute graph, preventing backpropagation of loss. For an MCTS-based agent, masking is essentially required as simulations into invalid state transitions corrupt value estimates and propagate poison up the tree. However, Huang and Ontañón's analysis is restricted to small, flat action spaces where the legal subset is a fixed function of a few features (MicroRTS). Chess presents a qualitatively harder case: legality depends on the global board configuration, and the function $A_{L(s)}$ is itself nontrivial to compute. This work investigates whether the conclusions of Huang and Ontañón (2020) generalize to environments where the rules constitute an algorithmic learning target.

Without masking, for any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space such that:

$$\pi^*(a|s) \to 0 \quad \text{for all } a \in \Big(A_{P(s)} \setminus A_{L(s)}\Big)$$

Under this paradigm, concepts such as pins and checkmating nets cease to be hardcoded environmental properties and become *emergent survival heuristics* within the value landscape.

2.4 Learning Dynamics

The unmasked, pseudo-legal regime forces the network to learn the rules of chess from scratch as a precondition for strategic evaluation, raising the prospect of non-monotonic learning dynamics. Power *et al.* (2022) identified the phenomenon of *grokking*, in which neural networks trained on small algorithmic datasets exhibit sharply delayed generalization long after overfitting the training set, mediated by weight-decay pressure. Chess is an algorithmic dataset embedded in a strategic MDP, showcasing the potential of analogous phase transitions.

Bengio *et al.* (2009) formalized *curriculum learning*, showing that ordering training data from simple to complex acts as a regularizer, guiding optimization toward superior local minima inaccessible through random shuffling. It is hypothesized a pseudo-legal environment serves implicitly as curriculum where an agent first receives rewards through piece kinematics and king capture before survival heuristics under A_L.

Sharma *et al.* (2017) introduced a framework for factoring discrete action spaces into compositional, basis-like components, enabling cross-action generalization among actions that share common factors. This principle is instantiated in the factored engine architecture, where each selection and destination shares a common output head.

2.5 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL (Huang and Ontañón (2020)), and Transformers have proven capable of modelling chess at a high level (Jenner *et al.* (2024)), the intersection of these domains is unexplored. No existing work isolates the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents, nor on the dynamics of rule internalization when those guardrails are removed.

In the unmasked, pseudo-legal regime, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that this configuration will exhibit non-monotonic learning dynamics: an initial regime of high illegal-move proposals, followed by a sharp convergence toward $A_{P(s)}$ as kinematics are internalized, acting as an emergent curriculum before strategic play develops in $A_{L(s)}$.

This project addresses this gap by training four distinct configurations across two orthogonal axes: Action Space Constraints (Legal vs. Pseudo-Legal) and Logit Masking (Masked vs. Unmasked). By utilizing a factored two-pass autoregressive encoder, this research provides a novel, quantitative analysis of how rule internalization and action space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

3.1 System Architecture Overview

The system is designed as a closed-loop reinforcement learning pipeline, decoupling the environment transition dynamics from the neural network's internal representations. The architecture consists of three primary components: a custom deterministic environment simulator supporting dual rulesets, a two-pass autoregressive Transformer encoder, and a bipartite MCTS algorithm. The system operates via self-play, generating trajectories to optimize the network.

Each component is designed such that the experimental axes can be toggled on or off independently without altering any other part of the system. This is essential to the validity of the comparison: the four configurations differ only in the two boolean flags governing their respective axes, with all hyperparameters held constant.

3.2 Functional and Non-Functional Requirements

The methodology imposes four hard requirements on the implementation, each of which constrains the design space from Section 3.3 to Section 3.7.

1. **Configuration Parity:** The four cells of the experimental matrix must differ only in the two boolean flags governing rule set and masking. No code path may exist that is reachable in one configuration but not another, with the sole exception of the masking step itself. This rules out per-configuration training scripts and motivates the unified `forward_classification` implementation in Section 4.4.2.
2. **Self-Play Throughput:** Multiple full training runs under a single GPU budget requires massive throughput. This rules out cache-hostile reference-counted tree structures and dynamic move-list allocation (heap pressure), motivating arena allocation (Section 4.3.1) and stack-allocated move lists (Section 4.2.2).
3. **Reproducibility:** Self-play is intrinsically stochastic, but every source of randomness must be seedable. Dirichlet noise (David Silver, Schrittwieser, *et al.* (2017)), temper-

ature sampling, replay sampling, and weight initialization all consume from seeded RNGs.

4. **Auditability:** Observed learning dynamics must be attributable to the network rather than to engine artefacts. This requires the move generator to be independently testable against a reference (Section 4.6.2) and the masking boundary (Section 4.2.2) to be unambiguous in code.

3.3 Environment Formulation and State Representation

3.3.1 Dual Ruleset MDPs

The **Legal Environment E_l** adheres to standard FIDE rules. The action space ($A_{l(s)}$) is strictly contained to moves that do not leave the king in check. The terminal reward $R(s, a)$ is evaluated upon Checkmate, Stalemate, or standard draw conditions. The **Pseudo-Legal Environment E_P** relaxes the action space to $A_{P(s)}$, permitting all geometrically valid piece movements regardless of king safety. The terminal condition shifts from checkmate to the explicit king-capture event K_{cap}. FIDE draw conditions (e.g., insufficient material, fifty-move rule) are retained to guarantee finite rollouts. In E_P, stalemate is redefined: if a king is not in check but the agent possesses no pseudo-legal moves (e.g., all pieces are physically blocked), the state evaluates as a draw.

3.3.2 State Canonicalization and Tensor Formulation

To enforce symmetric learning and halve the state space, all board states are canonicalized to the perspective of the side-to-move (David Silver, Hubert, *et al.* (2017)). If it is Black's turn, the board, and castling rights are geometrically flipped. The state s is mapped to a spatial tensor representation X and a scalar meta-tensor M, the exact dimensionalities of which are detailed in Section 4.4.1.

To explicitly measure spatial reasoning, the system factors the joint probability of a move $P(a|s)$ into an autoregressive sequence: $P(a|s) = P(\text{from}|s) \times P(\text{to}|s, \text{from})$. The network executes two forward passes using shared weights:

1. **Pass 1 (Piece Selection)**: The network evaluates the board state with the selected square plane zeroed out, outputting a policy distribution over the 64 squares to select the origin square (*from*). The value head will evaluate the board state.
2. **Pass 2 (Destination Selection)**: The origin square is encoded into the 14th plane of the spatial tensor. The network re-evaluates the state, outputting a policy distribution over the 64 squares to select the destination (*to*). The value head will evaluate the position given a piece to move, effectively evaluating the viability of the selected piece.

The factored $64 \to 64$ output lacks a third dimension for promotion piece selection. Promotions are instead handled via the search topology: when a pawn destination on the back rank is expanded, the corresponding edge is replaced by four sub-edges (Q, R, B, N), equally dividing the prior probability. This delegates under-promotion to the search tree, where the additional branching factor is amortized over many simulations.

3.3.3 Network Topology

The model uses a Transformer Encoder architecture without convolutions (Dosovitskiy *et al.* (2020)). The spatial tensor X is linearly projected to d_{model}, and $2D$ learned positional embeddings are added to retain geometric context. The meta-tensor M is linearly

projected and concatenated as a pseudo-[CLS] token, resulting in a sequence length of 65 (Monroe and Chalmers (2024)).

The model is symmetrically scaled as a variant of ChessTransformer (Monroe and Chalmers (2024)): $n_{\text{heads}} = 8$, $n_{\text{layers}} = 8$, $d_{\text{model}} = 8 \times 64 = 512$, $d_{\text{ff}} = 4 \times d_{\text{model}}$, meeting a middle ground between CF-6M and CF-240M.

3.3.4 Output Heads

The final latent representation is routed into two distinct heads (Mnih *et al.* (2016)):

1. **Policy head:** A linear projection of the 64 spatial tokens to $\mathbb{R}^{64}$, producing unnormalized logits over square selection.
2. **Value head:** A linear projection of the [CLS] token to $\mathbb{R}^3$, producing W/D/L logits. In Pass 1 this evaluates the raw state; in Pass 2 it evaluates the state conditional on the selected origin square.

3.4 Bipartite Monte Carlo Tree Search

3.4.1 Topology

To accommodate the two-pass autoregressive policy, the search tree alternates between two node types:

- **Selection Nodes (N_{select}):** Represent a board state. Edges represent the choice of an origin square.
- **Action Nodes (N_{move}):** Represent a board state plus a selected origin square. Edges represent the destination square; transitioning the environment to a new N_{select} node.

A complete move thus traverses a (Selection, Action) pair. This bipartite structure explicitly mirrors the models autoregressive nature, physically separating the two failure modes of factored policy in the search tree.

3.4.2 Selection and PUCT

During traversals, edges are selected using a modified Predictor + Upper Confidence Bound applied to Trees (PUCT) algorithm (David Silver, Hubert, *et al.* (2017)). To mitigate draw-seeking behaviour, a contempt factor is baked into the empirical action value $Q(s, a)$.

$$
\begin{aligned}
Q(s,a) &= W - L - 0.05D \\
U(s,a) &= c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)} + 10^{-8}}{1 + N(s,a)} \\
\text{PUCT}(s,a) &= Q(s,a) + U(s,a)
\end{aligned}
$$

where W, D, L are the mean W/D/L value estimates accumulated on the edge, $P(s, a)$ is the prior, and $N(s)$, $N(s, a)$ are the parent and edge visit counts respectively.

3.4.3 Exploration Mechanisms

Exploration is injected through two mechanisms. Dirichlet noise is applied exclusively to the root prior, with $P(s, a) \leftarrow (1 - \varepsilon)P(s, a) + \varepsilon \cdot \eta$, with $\eta \sim \text{Dir}(\alpha)$, $\alpha = 0.3$, and $\varepsilon = 0.25$ (David Silver, Schrittwieser, *et al.* (2017)). At the root, the move played is sampled in proportion to $N(s, a)^{\frac{1}{t}}$, adapting the David Silver, Hubert, *et al.* (2017) method by scaling

t dynamically with ply count and remaining material, transitioning from exploratory in the opening to deterministic in endgames.

3.4.4 Simulation Integrity and the Masking Boundary

Invalid state transitions will corrupt value propogation throughout the search tree. To maintain simulation integrity, action masking is used indiscriminantly by the orchestrator to ensure the search space involves only valid squares. The experimental distinction lies in whether action masking is applied to the network's raw logits during loss calculation. In masked configurations, illegal logits are clamped to -10^9 prior to softmax, training the model within a distribution restricted to valid squares. In unmasked configurations, the policy loss is computed over the raw 64 square distribution, punishing the model for assigning weight to invalid squares. The training signal still differs in whether the network is shielded from or penalized for invalid predictions, despite MCTS rollouts being strictly contained to the valid action space.

3.5 Reinforcement Learning Pipeline

3.5.1 Self-Play and Replay Buffer

Trajectories are generated via highly parallelized self-play. Samples are stored in a FIFO replay buffer of capacity $2^{19} = 524288$. To prevent infinite games, rollouts are forcibly terminated as draws after 400 plies, or whenever the MCTS root value estimate for a draw exceeds 0.95, relaxed to 0.75 after 60 plies (David Silver, Hubert, *et al.* (2017)).

3.6 Optimization and the Self-Annealing Loss

The network is optimized with AdamW (Loshchilov and Hutter (2017)) ($\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay 10^{-4}) under a Noam learning-rate schedule (Vaswani *et al.* (2017)) with factor 0.01 and 4000 warm-up steps. The value head is bootstrapped against the MCTS root-value distribution rather than the eventual game outcome (TD-like) (Sutton (1988)) for computational feasibility.

The principal methodological contribution lies in the loss function. For a sample (s, π, z) with policy prediction p and value prediction v, the loss is:

$$L = (1+\lambda) \cdot \underbrace{\left(-\sum_a \pi(a|s) \log p(a|s) \right)}_{L_{\text{policy}}} + (1-\lambda) \cdot \underbrace{\left(-\sum_i z_i \log v_i \right)}_{L_{\text{value}}}$$

where $\lambda \in [0, 1]$ is the mean probability mass that the network has assigned to illegal moves over the most recent batch of leaf expansions.

This acts as a strict mathematical curriculum. When the network proposes many illegal moves $\lambda \approx 1$, the policy loss weight approaches 2 and the value loss weight approaches 0. The network is forced to learn piece kinematics before state evaluation. As the model internalizes piece kinematics, the weights balance to $1:1$. This approach to curriculum learning is unique to the unmasked configuration, as λ is dependent on illegal move weights (λ is naturally zero in masked configurations).

3.7 Experimental Design and Evaluation Metrics

3.7.1 The Configuration Matrix

The configurations form a 5-cell matrix:

- **Legal + Masked (Control):** Standard AlphaZero paradigm.
- **Legal + Unmasked (Self-Annealing):** Network must learn piece kinematics before strategy.
- **Legal + Unmasked (Fixed Ratio):** Ablation study removing λ to isolate the effect of the self-annealing curriculum.

3.7.2 Quantitative Metrics

The learning dynamics of each configurations are captured throughout training in four metrics:

- **Average Illegal Probability:** The total policy weight attributed to invalid squares.
- **Average Game Length:** Measures in plies, indicating the transition from random walks to strategic, prolonged play.
- **Win/Draw Ratios**: Tracking the emergence of decisive outcomes versus stalemates or repetitions.
- **Value Loss Convergence**: Measures the accuracy of the network's state evaluation over time.

3.8 Discussion of Design Challenges

Three design problems were materially harder than they first appeared, and each shaped the final system in ways worth recording.

TODO

Chapter 4

Implementation Details

This chapter details the software architecture and engineering paradigms utilized to construct the self-play reinforcement learning system. The implementation translates the methodology into a high-throughput, reproducible execution environment optimized for three criteria: clean experimental toggling, sufficient throughput to complete four training runs within the project timeframe, and system auditability to isolate learning dynamics from engine artifacts.

The architecture comprises a custom bitboard-based chess engine, a bipartite MCTS utilizing flat arena allocators, and a shared-weight Transformer encoder implemented via the Burn deep learning framework (Simard *et al.* (2024)). All components are parallelized to bridge theoretical design with optimized execution.

4.1 Software Stack and Overview

The system is implemented entirely in Rust (The Rust Project Developers (2026)). The choice of Rust over C++ or Python is driven by the strict requirements of high-throughput MCTS. Python's Global Interpreter Lock (GIL) precludes true multithreading, forcing reliance on multi-processing which incurs prohibitive IPC serialization overhead for tree search. While C++ offers raw performance, complex concurrent tree mutations frequently introduce data races. Rust's ownership model and strict aliasing rules guarantee thread safety at compile time, enabling "fearless concurrency."

Neural network operations are executed via the Burn framework (Simard *et al.* (2024)). Unlike PyTorch's C++ bindings (LibTorch), which carry heavy binary bloat and tie execution to NVIDIA CUDA, Burn provides a backend-agnostic autograd module. By utilizing Burn's WGPU backend, the engine compiles to a standalone binary capable of executing hardware-accelerated tensor operations across CUDA, Metal, and Vulkan without environment configuration.

The self-play pipeline (Algorithm 1) is designed for maximum throughput via lock-free concurrency. At the initialization of each training iteration, B parallel game instances are spawned. As each MCTS instance encapsulates its own memory arenas, the Rayon

library's `par_iter_mut()` is utilized to distribute tree traversals across all available CPU cores without mutex contention.

Reinforcement Learning Loop

Require: Model weights θ, Replay Buffer D (capacity 2^{19}), Batch size B
Initialise B independent MCTS instances $\{T_1, ..., T_B\}$
while training **do**
  for step = 1 **to** steps_per_iteration **do**
    for i = 1 **to** simulations_per_move **do**
      parallel for each tree T in $\{T_1, ..., T_B\}$ **do**
        Traverse T to find leaf node n
      end
      Synchronise threads
      Batch inputs from all leaves and execute forward pass(θ)
      parallel for each tree T **do**
        Expand leaf n using network outputs
        Backpropagate value estimates to root
      end
    end
    parallel for each tree T **do**
      Select action $a \sim \pi(a|s)$ based on visit counts
      Store transition (s, π, z) in D
      Advance environment state
      if terminal state reached **then** reset T
    end
  end
  if $|D| >$ min_samples **then**
    for epoch = 1 **to** gradient_steps **do**
      Sample mini-batch from D
      Compute composite loss $L(\theta)$
      Update θ via AdamW
    end
  end
end

4.2 Chess Engine

The environment is a custom chess engine optimized specifically for MCTS rollouts. State representation is strictly packed, and move generation relies on compile-time precomputed masks to minimize branching and minimize CPU cache misses.

4.2.1 Bitboards

Alternative state representations, such as 8×8 mailbox arrays, require branching loops to evaluate sliding piece attacks. To maximize throughput, the board state is instead encoded using Bitboards (Chess Programming Wiki (2024a)). The `ChessBoard` struct uses a `[[Bitboard; 6]; 2]` array, representing the 6 piece types across 2 colors, alongside 3 aggregate occupancy bitboards (White, Black, All). The struct is annotated `#[repr(align(64))]` ensuring alignment with cache-line boundaries, preventing false sharing and making move generation cache-friendly under aggressive parallel access. Hardware intrinsics (`trailing_zeros` and `leading_zeros`) are used for $O(1)$ piece iteration and bit isolation.

4.2.2 Move Generation

The engine employs a two-step deferred verification architecture for move generation:

Precomputed Lookup Tables: Knight, King, Pawn, Rook, Bishop, and Rook/Bishop direction masks, together with a 64×64 `BETWEEN` table, are constructed via `const fn` and embedded directly in the binary as `pub const` arrays. This elides the static initialization phase entirely and allows the compiler to fold lookups into immediate operands where it can prove the index.

Sliding-Piece Move Generation: Sliding pieces are generated by AND-ing each direction mask with the global occupancy and isolating the first blocker via LSB or MSB depending on the direction (Chess Programming Wiki (2024b)):

```
let to_sq = if i < 2 { blockers.pop_msb() } else { blockers.pop_lsb() };
```

The blocker square is included in the ray if it is occupied by an enemy piece (a capture), and the precomputed `BETWEEN[from][blocker]` mask supplies the squares strictly between origin and blocker; the viable destination squares.

Stack-Allocated Move Lists: Generated moves are written to a stack-allocated `ArrayVec<ChessMove, 128>`, capping the branching factor to eliminate heap overhead while maintaining a statistically sufficient window for over 99% of chess positions.

Deferred Legality Verification: Rather than implementing computationally expensive pin-detection logic during generation, legality is verified post-hoc. The `is_legal()` method copies only the necessary board state, applies the pseudo-legal move, and evaluates if the resulting state leaves the active King under attack via `is_square_attacked()`. This deferred approach significantly reduces branching complexity while maintaining high throughput.

4.2.3 Zobrist Hashing

State repetition detection requires hashing board states. Cryptographic hashes (e.g., SHA-256) are computationally prohibitive, and full-state serialization is cache-inefficient. Therefore, Zobrist hashing Zobrist (1970) is implemented. A static table of 64-bit pseudo-random keys (`ZobristKeys`) is lazily initialized using Rust's `OnceLock`. The hash is computed in $O(P)$ time, where P is the number of active pieces, by XOR-ing the keys corresponding to the current board state, castling rights, en-passant file, and side-to-move. While incremental $O(1)$ hashing is theoretically optimal for sequential play, the $O(P)$ approach was selected to eliminate complex state-tracking across independent

MCTS nodes. Profiling indicates this computational cost is negligible relative to network inference (Section 4.6.1).

4.3 Bipartite Monte Carlo Tree Search

A standard MCTS represents a complete action as a single edge between two states. To map the search topology onto the network's two-pass autoregressive policy, the search tree is implemented as a bipartite graph alternating `PieceSelect` and `PieceMove` nodes (Section 3.4).

4.3.1 Arena Allocators

To avoid memory fragmentation and pointer-chasing overhead typically associated with recursive tree structures, the MCTS utilizes flat, index-based arena allocators (Hanson (1988)):

```
struct Arena<T> {
    buffer:   Vec<T>,
    freelist: Vec<usize>,
}

struct Mcts {
  node_arena: Arena<MctsNode>,
  edge_arena: Arena<MctsEdge>,
  position_arena: Arena<ChessPosition>,
  dead_nodes: Vec<usize>,
  ...
}
```

Nodes reference child edges and associated board states via `usize` indices. A dedicated `position_arena` is implemented to prevent deduplication of board state over consecutive `PieceSelect` and `PieceMove` nodes.

While Hanson's original arenas are deallocated in bulk, this implementation incorporates slab-style object reuse (Bonwick (1994)) to handle the high churn of MCTS subtrees. When subtrees are abandoned (e.g. after a move is chosen), their indices are returned to the freelist. By maintaining a freelist within the arena, abandoned node indices are recycled without requiring a full heap deallocation. Bulk edge insertion (`push_block`) searches for contiguous free slots. A garbage-collection pass recursively frees all nodes, edges, and board positions of abandoned subtrees. This keeps RAM proportional to the active search tree, enabling larger batch sizes.

4.3.2 Node Expansion, Promotions, and Terminals

Leaf expansion (Algorithm 2) transitions the state machine. Expanding a `PieceSelect` edge generates a `PieceMove` node without altering the underlying board state. Expanding a `PieceMove` edge applies the move, computes the new Zobrist hash (Zobrist (1970)), evaluates terminal states (including threefold repetition via path history), and generates a new `PieceSelect` node.

Promotions are handled natively by the bipartite structure (Section 3.3.2). When a pawn-to-back-rank destination is expanded, the MCTS intercepts the spatial prior p and distributes it uniformly ($p/4$) across four discrete edges (Q, R, B, N), each carrying the

corresponding promotion. This enables the model to handle underpromotions through search topology rather than increased output dimensionality.

BIPARTITE LEAF EXPANSION

Require: Leaf edge e, Network prior probabilities P, Search path history H
Let N_{parent} be the parent node of e
if N_{parent} is of type PieceSelect **then**
    Create N_{child} of type PieceMove
    Set N_{child}.selected_square = e.target_square
    Link e to N_{child}
else if N_{parent} is of type PieceMove **then**
    $s' \leftarrow$ Apply move(N_{parent}.selected_square, e.target_square)
    if Zobrist(s') appears ≥ 2 times in H **then**
        Create N_{child} as Terminal(Draw)
    else if is_terminal(s') **then**
        Create N_{child} as Terminal(Win/Loss)
    else
        Create N_{child} of type PieceSelect representing s'
    end
    Link e to N_{child}
end
if N_{child} is not Terminal **then**
    for each legal destination square i **do**
        if move is pawn promotion **then**
            Create 4 edges (Q, R, B, N) with prior = $P[i]/4$
        else
            Create 1 edge with prior = $P[i]$
        end
        Attach edge(s) to N_{child}
    end
end

The fifty-move rule has been shortened to forty full moves (eighty plies) to bound maximum episode length and reduce MCTS memory footprint.

4.3.3 Edge Selection and Value Propagation

PUCT scoring uses the contempt-augmented variant derived in Section 3.4.2:

- $Q_e = W_e - L_e - 0.05D_e$
- $U_e = c_{\text{puct}} \cdot p_e \cdot \frac{\sqrt{N(s)+10^{-8}}}{1+N(s,a)}$

The selected edge maximizes $Q_e + U_e$. On leaf evaluation, the value vector $[W, D, L]$ is back-propagated up the path. The vector is inverted ($[W, D, L] \leftarrow [L, D, W]$) if the current side-to-move $\neq$ leaf side-to-move. 10^{-8} is added to the exploitation numerator to ensure

non-zero exploration of unvisited nodes. The path is cleared after each simulation for the next traversal.

4.3.4 Mask Integration and Prior Renormalization

The network outputs a probability distribution over all 64 squares in unmasked configurations. To produce a valid prior for PUCT, the MCTS computes the illegal mass $\lambda = \sum_{i \notin A_X(s)} p_i$ and renormalizes the legal priors:

$$p'_i = \frac{p_i}{1 - \lambda} \quad \forall i \in A_X(s)$$

where $A_{X(s)}$ denotes the active configuration (A_L or A_P). The same λ is surfaced to the loss function as the curriculum coefficient described in Section 4.5.3.

4.4 Neural Network Architecture

The `ChessTransformer` is a shared-weight encoder mapping the spatial tensor and the meta-vector to a policy distribution and a W/D/L estimate.

4.4.1 Tensor Construction

Inputs are canonicalized to the side-to-move via `flip_board()` before being written into flat `Vec<f32>` slices and reshaped through Burn's `TensorData` API. This guarantees a contiguous memory layout before the host-to-device transfer, avoiding allocation churn. Tensor shapes are as follows:

- **Spatial tensor ($\boldsymbol{X} \in \mathbb{R}^{64\times14}$):** 12 piece planes, 1 en-passant plane, 1 selected-square plane (zero on the first pass, the highlighted origin square on the second).
- **Meta-tensor ($\boldsymbol{M} \in \mathbb{R}^{5}$):** 4 binary castling rights and 1 continuous half-move clock.

4.4.2 Encoder and Output Heads

As stated in Section 3.3.3, the 64 spatial vectors are linearly projected to $d_{\text{model}} = 512$, and learnable rank/file embeddings of size 8 each are summed and broadcast across the grid before being added to the spatial token sequence. The meta-vector is projected and concatenated as a pseudo-[CLS] token at index 64, yielding a $65 \times d_{\text{model}}$ sequence to be consumed by the encoder ($n_{\text{layers}} = 8$, $n_{\text{heads}} = 8$).

After the encoder, the sequence bifurcates:

- Tokens 0..63 pass through a linear policy head producing 64 spatial logits.
- Token 64 is passed through a linear value head producing the W/D/L logits.

The model's `forward_classification` method dynamically applies action masking based on configuration. In masked configurations, a boolean tensor identifies illegal squares and `mask_fill` clamps the corresponding logits to -10^9 before softmax. In unmasked configurations this step is bypassed and the cross-entropy is computed against the raw distribution, enabling the creation of the curriculum coefficient λ in Section 4.5.3.

4.5 Training Pipeline

The training pipeline orchestrates data generation, buffer management, and gradient updates, with each stage engineered to minimize idle time on either CPU or GPU.

4.5.1 Value Head Bootstrapping

Before the self-play loop begins, the value head is trained against a small dataset of positions annotated with mate-in-N evaluations (`mate_evals.tsv`, Lichess (2026)). One hundred gradient steps are run with the loss ratio fixed at -1 to ignore policy output. This step is negligible in wall-clock cost while greatly improving search tree efficiency.

4.5.2 Batch Expansion and Synchronization

The interface between the CPU-bound MCTS and the GPU-bound inference is implemented in `expand_batch` (Algorithm 3). MCTS instances traverse independently in parallel until each has selected a leaf. The threads then synchronize, aggregating canonicalized input tensors and masks into a single batch. Following a unified forward pass, the resulting `NetworkLabels` (policy and value arrays) are scattered back to their respective MCTS instances for edge expansion and value backpropagation. This single-batch boundary is the only synchronization point in the entire self-play loop, scaling compute cost with batch size so long as the GPU has memory to spare.

Batched Inference and Prior Renormalization

Require: Set of leaf nodes $\{n_1, ..., n_B\}$, Configuration C

$X_{\text{batch}}, M_{\text{meta}} \leftarrow$ Extract spatial and meta tensors from $\{n_1, ..., n_B\}$
$M_{\text{mask}} \leftarrow$ Generate legal move boolean masks for all $b \in B$
if C.masked is True **then**
  $P_{\text{raw}}, V_{\text{raw}} \leftarrow \text{TransformerForward}(X_{\text{batch}}, M_{\text{meta}}, M_{\text{mask}})$
else
  $P_{\text{raw}}, V_{\text{raw}} \leftarrow \text{TransformerForward}(X_{\text{batch}}, M_{\text{meta}}, \text{None})$
end
for $b = 1$ **to** B **do**
  $\lambda \leftarrow 0$
  if C.masked is False **then**
    for each illegal square i in n_b **do**
      $\lambda \leftarrow \lambda + P_{\text{raw}[b]}[i]$
    end
    for each legal square j in n_b **do**
      $P_{\text{raw}[b]}[j] \leftarrow P_{\text{raw}[b]} \frac{[j]}{1-\lambda}$
    end
  end
  n_b.illegal_mass $\leftarrow \lambda$
  n_b.value $\leftarrow V_{\text{raw}[b]}$
  Populate n_b edges using $P_{\text{raw}[b]}$
end

4.5.3 The Self-Annealing Loss in Code

The composite loss defined in Section 3.6 is implemented via `log_softmax` followed by exact multiplication:

```
let policy_probs = log_softmax(policy_pred, 1);
let policy_loss = (target_policy * policy_probs).sum_dim(1).mean().neg();

let value_probs = log_softmax(value_pred, 1);
let value_loss = (target_value * value_probs).sum_dim(1).mean().neg();

(policy_loss * (1.0 + ratio)) + (value_loss * (1.0 - ratio))
```

The `log_softmax`-then-multiply formulation is mathematically equivalent to a softmax followed by KL divergence against soft targets, but avoids the numerical instability of computing the softmax explicitly when logits span a wide range. `ratio` is naturally zero in masked configurations, and set to zero in the fixed-ratio ablation configuration based on an `annealing` flag. This unified implementation recovers the AlphaZero objective, the self-annealing curriculum, and the fixed-ratio ablation without separate code paths.

4.6 Performance and Correctness

4.6.1 Throughput and Memory

Engine throughput is dominated by the batched forward pass of the Transformer, with tree search and arena garbage collection running concurrently behind it. At a batch size and simulation count of 256 each, steady-state resident memory sits at approximately 11 GB system RAM and 10 GB VRAM. CPU utilization remains at 7% while GPU (NVIDIA RTX 4000 Ada) utilization reaches 100%, confirming network inference as the primary system bottleneck.

4.6.2 Verification and Testing

Three independent layers of verification target the three independent failure modes of the system.

Move generation is tested against the standard perft position suite (Kiwipete, Position 3, Position 4, Position 5, plus the initial position), exercising en-passant, castling through check, promotion, and double-pawn-push edge cases.

Tensor shapes are verified at compile time through Burn's typed tensor API, eliminating tensor mismatch bugs. **Zobrist hashes** use 64-bit keys, providing cryptographic-grade collision resistance over the search depths encountered during training.

Assertions are used throughout the stages of MCTS rollouts to ensure correctness in simulations.

4.6.3 Reproducibility

All hyperparameters used across the training runs are summarized in Table 1. The same values are used for every cell of the experimental matrix; the only variables are the two boolean flags `legal` and `masked`.

Parameter	**Value**
Batch size	256
MCTS simulations / move	256
Replay buffer capacity	2^{19}
Gradient steps / iteration	256
Self-play steps / iteration	256
d_{model}	512
n_{layers}	8
n_{heads}	8
d_{ff}	2048
c_{puct}	1.25
Dirichlet α	0.3
Dirichlet ε	0.25
Contempt (draw penalty)	0.05
Optimiser	AdamW
β_1, β_2	0.9, 0.99
Weight decay	10^{-4}
LR schedule	Noam, factor 0.01, 4000 warmup
Max ply / game	400
Half-move limit	80 plies
Pretraining steps (mate eval)	100
Random seed	1234

Table 1: Hyperparameters held constant across all five configurations.

Chapter 5

Results

5.1 Experimental Setup

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of deep late-stage grokking.
- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Policy Bootstrapping:** Boot strap the policy head with uniform distributions of valid moves, before self play.
- **Value Bootstrapping:** Boot strap the value head with a robust suite of perfect knowledge (mate evals, endgame tablebases, certain stockfish evaluations).

Chapter 7

Statement of Ethics

Bibliography

Andrew, B. and Richard S, S. (2018) "Reinforcement learning: an introduction". Available at: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook 2ndEd.pdf.

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Bonwick, J. (1994) "The slab allocator: An object-caching kernel memory allocator," in *USENIX Summer*. Available at: https://people.eecs.berkeley.edu/~kubitron/courses/cs194-24-S14/hand-outs/bonwick_slab.pdf.

Chess Programming Wiki (2024a) "Bitboards."

Chess Programming Wiki (2024b) "Bitboards."

Dosovitskiy, A. *et al.* (2020) "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929* [Preprint].

Hanson, D.R. (1988) *Fast Allocation and Deallocation of Memory Based on Object Lifetimes*. Technical Report TR-191–88. Available at: https://www.cs.princeton.edu/techreports/1988/191.pdf.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

Lichess (2026) *Lichess Open Database*. Available at: https://database.lichess.org/.

Loshchilov, I. and Hutter, F. (2017) "Fixing Weight Decay Regularization in Adam," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1711.05101.

McCarthy, J. (1990) "Chess as the Drosophila of AI," in T.A. Marsland and J. Schaeffer (eds.) *Computers, Chess, and Cognition*. New York, NY: Springer New York, pp. 227–237.

McGrath, T. *et al.* (2022) "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, 119(47). Available at: https://doi.org/10.1073/pnas.2206625119.

Mnih, V. *et al.* (2016) "Asynchronous Methods for Deep Reinforcement Learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1602.01783.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

The Rust Project Developers (2026) "The Rust Programming Language."

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Sharma, S. *et al.* (2017) "Learning to Factor Policies and Action-Value Functions: Factored Action Space Representations for Deep Reinforcement learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1705.07269.

Silver, David, Hubert, *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

Silver, David, Schrittwieser, *et al.* (2017) "Mastering the game of Go without human knowledge," *Nature*, 550(7676), pp. 354–359. Available at: https://doi.org/10.1038/nature24270.

Simard, N. *et al.* (2024) "Burn: Deep Learning Framework in Rust." GitHub.

Sutton, R.S. (1988) "Learning to predict by the methods of temporal differences," *Machine Learning*, 3(1), pp. 9–44. Available at: https://doi.org/10.1007/BF00115009.

Vaswani, A. *et al.* (2017) "Attention Is All You Need," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1706.03762.

Vinyals, O. *et al.* (2019) "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, 575(7782), pp. 350–354. Available at: https://doi.org/10.1038/s41586-019-1724-z.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.

Zobrist, A.L. (1970) *A New Hashing Method with Application for Game Playing*. technical report 88. Madison, WI, USA. Available at: http://www.cs.wisc.edu/techreports/1970/TR88.pdf.